

Частное учреждение профессионального образования

«Высшая школа предпринимательства»

(ЧУПО «ВШП»)

ОТЧЕТ ПО ПРАКТИКЕ

по основной образовательной программе
среднего профессионального образования по специальности
09.02.07 «Информационные системы и программирование»

Вид практики (учебная, производственная, преддипломная):

Установленный по КУГ срок прохождения практики: с __.__.20__ г. по
___.20__ г.

Место прохождения практики (наименование организации):

Выполнил

студент

___-го курса

(подпись)

(фамилия, имя, отчество)

Руководитель от _____
образовательной _____
организации _____

(подпись) (ученая степень, фамилия, имя,
отчество)

(должность)

Руководитель от _____
предприятия _____

(подпись) (ученая степень, фамилия, имя,
отчество)

(должность)

Оценка _____
(прописью)

Дата сдачи отчета: __.__.20__ г.

Оглавление Введение.....	3
Массивы и связанные списки.....	4
Стек и очередь.....	9
Map, HashMap, хэш-функции и коллизии.....	11
Деревья и графы.....	13
Куча и приоритетные очереди.....	16
Заключение.....	19
Список источников.....	20
Приложение 1.Ссылка на репозиторий.....	21

Введение

Учебная практика УП.02 «Алгоритмы и вычислительная сложность» была выполнена в период с 1 по 19 декабря 2025 года по специальности «Информационные системы и программирование».

Основной целью практики стало углубление теоретических знаний в области структур данных и алгоритмов, а также формирование практических навыков их реализации, анализа эффективности и применения для решения типовых задач.

В соответствии с программой практики были определены следующие задачи:

Изучить асимптотическую нотацию и методы оценки вычислительной сложности.

Реализовать ключевые структуры данных: массивы, списки, стек, очередь, хэш-таблицы, деревья, графы, кучи.

Провести сравнительный анализ временной и пространственной сложности операций.

Применить реализованные структуры для решения задач (проверка скобок, калькулятор в ОПН, поиск островов и др.).

Оптимизировать решения с учётом выявленных узких мест.

Подготовить документацию и открытый репозиторий с исходным кодом.

В ходе практики было выполнено 16 заданий, охватывающих пять основных тематических блоков. Каждая реализация сопровождалась анализом сложности, тестированием и (в отдельных случаях) профилированием производительности.

Массивы и связанные списки

Статический массив

Был разработан класс `StaticArray`, обеспечивающий фиксированный объём памяти. Все операции сопровождались анализом трудоёмкости:

`pushBack` — добавление в конец: $O(1)$.

`pushFront` — вставка в начало с последующим сдвигом всех элементов: $O(n)$.

`insert(index, value)` — вставка по индексу: $O(n)$ в худшем случае.

`remove(index)` — удаление с линейным сдвигом: $O(n)$.

`find(value)` — линейный поиск: $O(n)$.

Результаты показали, что статический массив обеспечивает мгновенный доступ по индексу, но неэффективен при частых модификациях в начале или середине.

Задание 1. Статический массив

1. Реализовать функции:

- `pushBack`, `pushFront`,
- `insert(index, value)`,
- `remove(index)`, • `find(value)`.

2. Оценить трудоёмкость каждой операции (в комментариях).

Вывод:

Реализация статического массива с фиксированной ёмкостью позволила наглядно продемонстрировать фундаментальный компромисс между скоростью доступа и стоимостью модификации. Операции pushBack и прямой доступ по индексу выполняются за $O(1)$, что делает массив идеальным для чтения. Однако вставка или удаление в начале или середине требуют сдвига всех последующих элементов, что приводит к $O(n)$ временной сложности. Это подтверждает, что статический массив наиболее эффективен в сценариях с предопределённым объёмом данных и минимальным количеством изменений.

Задание 2. Динамический массив

Реализация динамического массива использует стратегию удвоения ёмкости при переполнении. Это обеспечивает амортизированную сложность вставки $O(1)$.

Эмпирические замеры при добавлении 100 000 элементов:

Статический массив: 0.0078 с

Динамический массив: 0.0107 с

Несмотря на небольшое замедление из-за периодического копирования данных, динамический массив предпочтителен, когда объём данных заранее неизвестен.

Вывод:

Реализация динамического массива с автоматическим расширением (стратегия $\times 2$) показала, как можно сохранить простоту массива, устранив ограничение фиксированного размера. Несмотря на небольшое

замедление при копировании элементов во время ресайза, амортизированная сложность вставки остаётся $O(1)$. Эмпирическое сравнение со статическим массивом подтвердило: динамический массив немного медленнее, но значительно удобнее в реальных условиях, где объём данных заранее неизвестен. Это делает его предпочтительным выбором для большинства прикладных задач.

Задание 3. Односвязный список

1.Реализовать:

- вставку в начало/конец,
- удаление по значению,
- поиск по значению,
- разворот списка in-place.

Сравнить операции вставки/удаления с массивом.

Реализованы операции:

- вставка в начало и конец,
- удаление и поиск по значению,
- разворот списка in-place.

Особенностью является $O(1)$ для вставки в начало и $O(n)$ для доступа к произвольному элементу. В отличие от массива, список не требует сдвига элементов при удалении, но теряет локальность данных и потребляет больше памяти.

Пример кода одной из функций:

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None
```

```
class LinkedList:
    def __init__(self):
        self.head = None
```

```
def insert_at_beginning(self, value):
    new_node = Node(value)
    new_node.next = self.head
    self.head = new_node
```

Вывод:

Односвязный список продемонстрировал альтернативный подход к хранению последовательных данных, в котором вставка и удаление в начале выполняются за $O(1)$ без сдвига элементов. Однако доступ к произвольному элементу требует линейного обхода — $O(n)$. Разворот списка in-place стал примером эффективного использования указателей для трансформации структуры без дополнительной памяти. Сравнение с массивом показало: списки предпочтительны при частых модификациях,

особенно в начале, но уступают в скорости случайного доступа и потреблении памяти.

Задание 4. Двусвязный список

1. Реализовать:

- вставку после произвольного узла,
- удаление узла без поиска “сначала”.

2. Реализовать итератор по двусвязному списку.

Реализованные функции:

- `insert_after(node, value)` — вставляет новый узел после указанного узла. Для этого изменяются ссылки соседних узлов, чтобы новый узел оказался сразу после заданного.
- `remove_node(node)` — удаляет указанный узел без предварительного поиска. При этом переназначаются ссылки соседних узлов, а если удаляется голова или хвост списка, обновляются ссылки на `head` и `tail`.
- Итератор (`__iter__`) — обеспечивает перебор всех элементов списка. Это позволяет использовать цикл `for` для прохода по всем значениям списка.

Пример кода одной из функций:

```
def insert_after(self, node, value):  
    if node is None:  
        return  
  
    new_node = node(value)
```

```
next_node = node.next

node.next = new_node
new_node.prev = node

if next_node:
    new_node.next = next_node
    next_node.prev = new_node
else:
    self.tail = new_node
```

Вывод:

Двусвязный список расширил возможности односвязного, добавив обратные ссылки, что позволило реализовать удаление узла без поиска (при наличии ссылки на него) и вставку после произвольного узла за $O(1)$. Итератор сделал структуру совместимой с циклами `for`, повысив удобство использования. Вывод: двусвязный список — оптимальное решение для сценариев, где часто требуется локальная модификация в середине последовательности, несмотря на увеличение памяти на 50–100% из-за хранения двух указателей.

Стек и очередь

Задание 5. Стек 1. Реализовать стек на:

- массиве,
- связном списке.

2. Используя стек, проверить корректность скобочной последовательности.

Стек был реализован двумя способами:

на основе динамического массива,

на основе односвязного списка.

Обе реализации поддерживают $O(1)$ для push и pop. Стек на массиве проще и быстрее благодаря лучшей локальности памяти. На его основе была решена задача проверки корректности скобочной последовательности, где каждая закрывающая скобка должна соответствовать последней незакрытой открывающей.

Вывод:

Реализация стека на массиве и списке подтвердила универсальность LIFO-принципа. Оба варианта обеспечивают $O(1)$ для push/pop, но стек на массиве выгоднее по локальности памяти и накладным расходам. Практическое применение — проверка корректности скобочной последовательности — наглядно продемонстрировало, как стек может отслеживать «глубину вложенности» и обеспечивать структурную валидацию. Это подтверждает, что стек — не просто структура, а мощный инструмент для синтаксического анализа и отката состояний.

Задание 6. Очередь

- Реализовать очередь на циклическом массиве.
- Реализовать очередь на двух стеках.

Реализация очереди

Рассмотрены два подхода:

Циклический массив — фиксированный объём, два указателя (*front*, *rear*). Все операции — $O(1)$, минимальные накладные расходы.

Очередь на двух стеках — демонстрирует, как FIFO можно эмулировать с помощью LIFO. Операции в среднем — $O(1)$, но в худшем случае — $O(n)$.

Вывод:

Сравнение двух реализаций очереди выявило их сильные стороны:

- Циклический массив — максимальная эффективность при фиксированном размере, $O(1)$ для всех операций, минимальное потребление памяти.
- Очередь на двух стеках — теоретический интерес, демонстрирующий, как FIFO можно реализовать через LIFO, но с $O(n)$ в худшем случае при перекладывании элементов.

Вывод: для реальных систем предпочтителен циклический массив, тогда как очередь на стеках полезна для обучения и понимания абстракций.

Задание 7. Задача “Калькулятор”

- Читать выражение в инфиксной форме.
- Используя стек, преобразовать в обратную польскую нотацию (ОПН).
- Вычислить результат.

Калькулятор в обратной польской нотации

Реализован алгоритм, преобразующий инфиксное выражение (например, $3 + 4 * 2$) в ОПН ($3\ 4\ 2\ *\ +$) с последующим вычислением.

Использование стека позволило корректно обрабатывать приоритеты операций и вложенность скобок.

Вывод:

Реализация калькулятора с преобразованием в обратную польскую нотацию (ОПН) стала ярким примером применения стека для управления приоритетами операций. Алгоритм Дейкстры («сортировочная станция») корректно обрабатывает скобки и арифметические приоритеты, а последующее вычисление ОПН — простой и надёжный процесс. Это задание показало, что стек упрощает разбор выражений, делая алгоритм масштабируемым (легко добавить новые операторы) и устойчивым к ошибкам синтаксиса.

Map, HashMap, хэш-функции и коллизии

Задание 8. Своя хэш-таблица

1. Реализовать:

- хэш-функцию для строк,
- метод разрешения коллизий (цепочки или открытая адресация),
- функции `put(key, value)`, `get(key)`, `remove(key)`.

2. Визуализировать состояние таблицы после серии вставок.

Хэш-таблицы и работа с коллизиями

Собственная хэш-таблица

Разработана хэш-таблица с:

хэш-функцией: $\text{hash}(s) = \sum(\text{ord}(c) * 31^{**i}) \% \text{size}$,

разрешением коллизий методом цепочек,

операциями put, get, remove.

Функция визуализации показывает распределение ключей по «корзинам», что помогает оценить равномерность хэширования.

Вывод:

Реализация хэш-таблицы с цепочками и собственной хэш-функцией подчеркнула ключевую роль качества хэширования. При хорошей функции операции put/get/remove выполнялись за $O(1)$ в среднем, а визуализация распределения по корзинам помогла оценить равномерность. Это задание доказало, что хэш-таблица — одна из самых эффективных структур для поиска, но её производительность напрямую зависит от хэш-функции, что делает её проектирование нетривиальной задачей.

Задание 9. Частотный словарь

1. Построить HashMap частот встречаемости слов в тексте.
2. Вывести топ-10 самых частых слов.
3. Сравнить время построения частотного словаря при:
 - плохой хэш-функции (например, всегда 1),
 - хорошей хэш-функции.

Частотный словарь

Реализован подсчёт частоты слов в тексте. Сравнение показало:

при хорошей хэш-функции: время $O(n)$,

при плохой (всегда возвращает 0): время $O(n^2)$ из-за длинных цепочек.

Это наглядно демонстрирует критическую важность качества хэш-функции.цепочках.

Вывод:

Сравнение времени построения частотного словаря с хорошей и плохой хэш-функцией дало убедительный результат: плохая функция (всегда возвращает 0) превратила хэш-таблицу в линейный список, увеличив сложность до $O(n^2)$. В то же время хорошая функция обеспечила $O(n)$. Это задание стало практическим доказательством: алгоритмическая сложность — не теория, а реальный фактор производительности, особенно при работе с большими объёмами данных.

Задание 10. Trie + HashMap: автодополнение

1. Реализовать Trie для хранения слов.
2. Реализовать поиск по префиксу: метод `autocomplete(prefix)`.
3. Используя HashMap + Trie:
 - хранить слова + их частоты,
 - предлагать подсказки в порядке убывания частоты

Комбинация Trie и HashMap позволила реализовать интеллектуальное автодополнение, учитывающее частоту слов. Trie обеспечивает быстрый поиск по префиксу за $O(m)$, где m — длина префикса, а HashMap хранит частоты для сортировки подсказок. Это решение оптимально для интерактивных систем ввода, таких как поисковые строки или клавиатуры смартфонов, где требуется мгновенный отклик и релевантность.

```
def autocomplete(self, prefix, limit=10):
    node = self.root
    for char in prefix:
        if char not in node.children:
            return []
        node = node.children[char]

    results = []
    self._collect_words(node, prefix, results)

    results.sort(key=lambda x: x[1], reverse=True)
    return [word for word, freq in results[:limit]]
```

Вывод:

Комбинация Trie и HashMap позволила реализовать интеллектуальное автодополнение, учитывающее частоту слов. Trie обеспечивает быстрый поиск по префиксу за $O(m)$, где m — длина префикса, а HashMap хранит частоты для сортировки подсказок. Это решение оптимально для

интерактивных систем ввода, таких как поисковые строки или клавиатуры смартфонов, где требуется мгновенный отклик и релевантность.

Деревья и графы

Задание 11. Бинарное дерево поиска (BST)

1. Реализовать: 14

- вставку,
- поиск,
- удаление,
- обходы: in-order, pre-order, post-order.

2. Проверить, является ли дерево сбалансированным.

Бинарное дерево поиска (BST)

Реализованы:

вставка, поиск, удаление (с обработкой трёх случаев),

три вида обхода: in-order, pre-order, post-order,

проверка сбалансированности (разница высот поддеревьев ≤ 1).

BST обеспечивает $O(\log n)$ для всех операций в среднем случае, но деградирует до $O(n)$ при несбалансированности.

Вывод:

Реализация BST показала, как можно хранить данные в отсортированном виде с поддержкой быстрого поиска, вставки и удаления за $O(\log n)$ в сбалансированном случае. Обходы (in/pre/post-order) позволили получить данные в разных порядках, что полезно для сериализации и анализа. Однако задание также выявило

главный недостаток BST — деградация до $O(n)$ при несбалансированности, что подтверждает необходимость использования самобалансирующихся деревьев (AVL, Red-Black) в продакшене.

Задание 12. Trie (углубление)

1. Добавить в Trie возможность:

- хранения слов целиком,
- подсчёта количества вариантов по префиксу,
- удаления слова.

Углублённая реализация Trie с поддержкой подсчёта количества слов по префиксу и удаления превратила структуру в полноценное хранилище строк. Счётчик префиксов у каждого узла позволил мгновенно отвечать на запросы вида «сколько слов начинаются с "app"?». Удаление с рекурсивной очисткой неиспользуемых узлов обеспечило эффективное управление памятью. Это делает Trie незаменимым для лексикографических задач и маршрутизации (например, в IP-адресах).

Задание 13. Графы

1. Хранение графов:

- матрица смежности,
- список смежности.

2. Реализовать алгоритмы:

- BFS,
- DFS,

- поиск кратчайшего пути в невзвешенном графе (BFS).

Реализация графов через матрицу и список смежности выявила их сферы применения:

Матрица — эффективна для плотных графов и проверки существования ребра за $O(1)$, но требует $O(n^2)$ памяти.

Список — экономичен для разреженных графов, используя $O(n + m)$ памяти.

Алгоритмы BFS и DFS оказались универсальными инструментами: BFS — для кратчайших путей в невзвешенных графах, DFS — для анализа связности и топологической сортировки. Это подтверждает, что выбор представления графа должен основываться на характере данных и задачи.

Вывод:

Реализация графов через матрицу и список смежности выявила их сферы применения:

- Матрица — эффективна для плотных графов и проверки существования ребра за $O(1)$, но требует $O(n^2)$ памяти.

- Список — экономичен для разреженных графов, используя $O(n + m)$ памяти.

Алгоритмы BFS и DFS оказались универсальными инструментами: BFS — для кратчайших путей в невзвешенных графах, DFS — для анализа связности и топологической сортировки. Это подтверждает, что выбор

представления графа должен основываться на характере данных и задачи.

Задание 14. Задача “Острова”

Дан двумерный массив 0/1. Найти количество “островов” (компонент связности).

Использовать DFS или BFS.

На двумерной матрице из 0 и 1 подсчитано количество компонент связности («островов»). Решение использует DFS: при обнаружении единицы запускается рекурсивный обход, «затопляющий» весь остров (заменяя 1 на 0), чтобы избежать повторного подсчёта.

Вывод:

Применение DFS для подсчёта компонент связности в двумерной матрице продемонстрировало мощь рекурсивного обхода в задачах пространственного анализа. Алгоритм «затопляет» каждый найденный остров, предотвращая повторный подсчёт, и работает за $O(n \cdot m)$ — оптимально для данной задачи. Это задание показало, что графовые алгоритмы применимы даже к неочевидным структурам, таким как изображения или карты.

Куча и приоритетные очереди

Задание 15. Куча

1. Реализовать бинарную мин-кучу:

- вставку,
- извлечение минимума,
- построение кучи из массива.

Проверить корректность свойств кучи после каждой операции.

В этом задании я реализовала бинарную мин-кучу — особую структуру данных, в которой каждый родительский элемент не превышает своих потомков. Куча представлена в виде массива, и порядок элементов поддерживается при каждой операции. Метод `insert` добавляет новый элемент в конец массива, а затем перемещает его вверх по дереву (`heapify_up`), пока не будет восстановлено свойство кучи.

```
def insert(self, value):  
    self.heap.append(value)  
    self._heapify_up(len(self.heap) - 1)  
    return self.is_valid_heap()
```

Метод `extract_min` выбирает минимальный элемент, находящийся в корне кучи. Он меняет элемент последним элементом массива и затем переносит его вниз (с помощью метода `heapify_down`), чтобы восстановить порядок элементов в куче.

```
def extract_min(self):
    if not self.heap:
        return None

    if len(self.heap) == 1:
        return self.heap.pop()

    min_val = self.heap[0]
    self.heap[0] = self.heap.pop()
    self._heapify_down(0)

    return min_val, self.is_valid_heap()
```

Мин-куча

Реализована бинарная мин-куча на массиве:

- ☐ insert — $O(\log n)$,
- ☐ extract_min — $O(\log n)$,
- ☐ build_heap — $O(n)$.

После каждой операции проверялось свойство кучи: значение родителя $\leq$ значений потомков.

Вывод:

Реализация мин-кучи на массиве подтвердила её свойства:

- Минимальный элемент всегда в корне,
- Вставка и извлечение за $O(\log n)$,

- Построение из массива за $O(n)$.

Проверка корректности после каждой операции обеспечила надёжность. Куча оказалась идеальной структурой для задач, где важен доступ к экстремуму, но не требуется полная сортировка.

Задание 16. Приоритетная очередь

1. Используя heap, реализовать:

- `push(value, priority)`,
- `pop()` — всегда возвращает элемент с минимальным приоритетом.

2. Применить к задаче:

- планирование задач (task scheduling),
- поиск k минимальных элементов массива.

На основе кучи создана очередь, где каждый элемент — пара (приоритет, значение). Операции:

`push(value, priority)` — вставка с учётом приоритета,

`pop()` — извлечение элемента с минимальным приоритетом.

Применение:

планирование задач — выполнение в порядке важности,

поиск k минимальных элементов — эффективнее полной сортировки при малых k .

Вывод:

Построение приоритетной очереди на основе кучи позволило реализовать управление задачами по приоритету. Применение к поиску

k минимальных элементов показало её преимущество перед полной сортировкой: при малых k сложность $O(n \log k)$ vs $O(n \log n)$. Это подтверждает, что приоритетная очередь — ключевой инструмент для оптимизации ресурсоёмких задач, таких как планирование, сортировка и алгоритмы на графах (например, Дейкстры).

Заключение

В ходе учебной практики УП.02 "Алгоритмы и вычислительная сложность" были успешно выполнены все поставленные задачи.

Реализованы 16 заданий, охватывающих основные структуры данных и алгоритмы:

1. Освоены методы оценки вычислительной сложности алгоритмов (Big O нотация).
2. Реализованы базовые структуры данных: массивы, списки, стеки, очереди.
3. Созданы более сложные структуры: хэш-таблицы, деревья, графы, кучи.
4. Разработаны алгоритмы для решения практических задач.
5. Проведен сравнительный анализ производительности различных реализаций.
6. Созданы визуализации для лучшего понимания работы алгоритмов.

Практика позволила закрепить теоретические знания, полученные в ходе обучения, и приобрести практические навыки программирования. Все реализованные структуры данных сопровождаются подробными комментариями о временной и пространственной сложности операций.

Результаты практики могут быть использованы:

- Как учебное пособие по алгоритмам и структурам данных

- Как основа для более сложных проектов
- Как справочный материал при подготовке к собеседованиям

Работа показала важность не только корректности реализации, но и понимания вычислительной сложности алгоритмов для создания эффективных программных решений.

Список источников

1. Кормен, Т. Алгоритмы: построение и анализ / Т. Кормен, Ч. Лейзерсон, Р. Ривест, К. Штайн. – 3-е изд. – М.: Вильямс, 2013. – 1328 с.
2. Макконнелл, Дж. Анализ алгоритмов. Активный обучающий подход / Дж. Макконнелл. – 3-е изд. – М.: Техносфера, 2009. – 416 с.
3. Скиена, С. Алгоритмы. Руководство по разработке / С. Скиена. – 2-е изд. – М.: БХВ-Петербург, 2011. – 720 с.
4. Python Documentation [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/>
5. GeeksforGeeks. Data Structures and Algorithms [Электронный ресурс]. – Режим доступа: <https://www.geeksforgeeks.org/data-structures/>
6. LeetCode. Algorithms Problems [Электронный ресурс]. – Режим доступа: <https://leetcode.com/problemset/algorithms/>
7. Wikipedia. Big O notation [Электронный ресурс]. – Режим доступа: https://en.wikipedia.org/wiki/Big_O_notation
8. Visualgo. Data Structures Visualization [Электронный ресурс]. – Режим доступа: <https://visualgo.net/en>

Приложение 1. Ссылка на репозиторий

GitHub репозиторий с полным кодом всех реализаций:

<https://github.com/ulianka69/UP02-Algorithms-Practice>